

MULTI-THREADING

Parallelism (Subprograms run simultaneously for faster programs) VS. **Concurrency** (interleaved execution of programs for simpler programs)

Process: Program under execution, own address space
(heavyweight, Proc: process isolation and responsiveness, Cms: interprocess communication overhead, expensive in creation, slow context switching and process termination).

Thread: Parallel sequence within a process. Sharing the same address space and registers (lightweight because most of the overhead already happened in the process creation).

Context switch: Required when changing threads.
Synchronous (Thread waiting for condition) OR **Asynchronous** (Thread gets released after defined time)

Multitasking: **Cooperative** (Threads must explicitly initiate context switches, scheduler can't interrupt) OR **preemptive** (scheduler can asynchronously interrupt thread via timer interrupt)
JVM Thread Model: JVM is a process in the OS. It runs as long as threads are running (Exception: threads marked as daemon with `setDaemon(true)` will not be waited upon). Threads are realized by the thread class and the interface **Runnable**. Code to be run in a Thread is within a overridden run().

Process: Program under execution, own address space (heavyweight, Proc: process isolation and responsiveness, Cms: interprocess communication overhead, expensive in creation, slow context switching and process termination).

Thread: Parallel sequence within a process. Sharing the same address space, but separate stack and registers (lightweight because most of the overhead already happened in the process creation).

Multi-threads: Changes made by one thread to shared resources will be seen by other threads.
Context switch: Required when changing threads.

Synchronous (Thread waiting for condition) OR **Asynchronous** (Thread gets released after defined time)

Multitasking: **Cooperative** (Threads must explicitly initiate context switches, scheduler can't interrupt) OR **preemptive** (scheduler can asynchronously interrupt thread via timer interrupt)
JVM Thread Model: JVM is a process in the OS. It runs as long as threads are running (Exception: threads marked as daemon with `setDaemon(true)` will not be waited upon). Threads are realized by the thread class and the interface **Runnable**. Code to be run in a Thread is within a overridden run().

create + start a thread
// explicit Runnable implementation
// class myRunnable implements Runnable {
// @Override
// public void run() {
// // thread behavior +/
// }
// }
var myThread = new Thread(new myRunnable());
myThread.start();
// named function
var myThread = new Thread(() -> {
// namedFunction();
});
// anonymous function (lambda)
var myThread = new Thread(() -> {
// thread behavior +/
});
// Or lambda lambda with + arrow

Multi-Thread Examples Java
// no synchronization
// class myRunnable implements Runnable {
// @Override
// public void run() {
// // thread behavior +/
// }
// }
var myThread = new Thread(new myRunnable());
myThread.start();
// named function
var myThread = new Thread(() -> {
// namedFunction();
});
// anonymous function (lambda)
var myThread = new Thread(() -> {
// thread behavior +/
});
// Or lambda lambda with + arrow

Thread Methods Thread.x (Java 8)
start() (Start), join() (Join) wait till thread finished, timeout: join(100), sleep(ms) (Sleep) (ms) join-sleep + Timed_Waiting state, wait() gives processor-lock free, yield() scheduler can release processor, to Ready state, does not release lock, setDaemon(true) (isBackground = true);

Java Thread States: Blocked (blocked and waiting for a monitor lock), New (not yet started), Runnable (Ready to run or running), Terminated, Timed_Waiting (waiting with a specified waiting time: Thread.sleep(ms)/Thread.join(ms)), Waiting

Interrupts: Interrupted with `Thread.interrupt()` If thread is in `sleep()`, `wait()` or `join()` methods, a `InterruptedException` is thrown. Otherwise flag is set, check with: `interrupted()` or `isInterrupted()`
Exceptions: Exceptions thrown in run() can't be propagated to the Main thread. The exception needs to be handled within the code executed on the thread.

Thread Methods Thread.x (Java 8)
start() (Start), join() (Join) wait till thread finished, timeout: join(100), sleep(ms) (Sleep) (ms) join-sleep + Timed_Waiting state, wait() gives processor-lock free, yield() scheduler can release processor, to Ready state, does not release lock, setDaemon(true) (isBackground = true);

Java Thread States: Blocked (blocked and waiting for a monitor lock), New (not yet started), Runnable (Ready to run or running), Terminated, Timed_Waiting (waiting with a specified waiting time: Thread.sleep(ms)/Thread.join(ms)), Waiting

Interrupts: Interrupted with `Thread.interrupt()` If thread is in `sleep()`, `wait()` or `join()` methods, a `InterruptedException` is thrown. Otherwise flag is set, check with: `interrupted()` or `isInterrupted()`
Exceptions: Exceptions thrown in run() can't be propagated to the Main thread. The exception needs to be handled within the code executed on the thread.

Thread Methods Thread.x (Java 8)
start() (Start), join() (Join) wait till thread finished, timeout: join(100), sleep(ms) (Sleep) (ms) join-sleep + Timed_Waiting state, wait() gives processor-lock free, yield() scheduler can release processor, to Ready state, does not release lock, setDaemon(true) (isBackground = true);

Java Thread States: Blocked (blocked and waiting for a monitor lock), New (not yet started), Runnable (Ready to run or running), Terminated, Timed_Waiting (waiting with a specified waiting time: Thread.sleep(ms)/Thread.join(ms)), Waiting

Interrupts: Interrupted with `Thread.interrupt()` If thread is in `sleep()`, `wait()` or `join()` methods, a `InterruptedException` is thrown. Otherwise flag is set, check with: `interrupted()` or `isInterrupted()`
Exceptions: Exceptions thrown in run() can't be propagated to the Main thread. The exception needs to be handled within the code executed on the thread.

Thread Methods Thread.x (Java 8)
start() (Start), join() (Join) wait till thread finished, timeout: join(100), sleep(ms) (Sleep) (ms) join-sleep + Timed_Waiting state, wait() gives processor-lock free, yield() scheduler can release processor, to Ready state, does not release lock, setDaemon(true) (isBackground = true);

Java Thread States: Blocked (blocked and waiting for a monitor lock), New (not yet started), Runnable (Ready to run or running), Terminated, Timed_Waiting (waiting with a specified waiting time: Thread.sleep(ms)/Thread.join(ms)), Waiting

Interrupts: Interrupted with `Thread.interrupt()` If thread is in `sleep()`, `wait()` or `join()` methods, a `InterruptedException` is thrown. Otherwise flag is set, check with: `interrupted()` or `isInterrupted()`
Exceptions: Exceptions thrown in run() can't be propagated to the Main thread. The exception needs to be handled within the code executed on the thread.

Thread Methods Thread.x (Java 8)
start() (Start), join() (Join) wait till thread finished, timeout: join(100), sleep(ms) (Sleep) (ms) join-sleep + Timed_Waiting state, wait() gives processor-lock free, yield() scheduler can release processor, to Ready state, does not release lock, setDaemon(true) (isBackground = true);

Java Thread States: Blocked (blocked and waiting for a monitor lock), New (not yet started), Runnable (Ready to run or running), Terminated, Timed_Waiting (waiting with a specified waiting time: Thread.sleep(ms)/Thread.join(ms)), Waiting

Interrupts: Interrupted with `Thread.interrupt()` If thread is in `sleep()`, `wait()` or `join()` methods, a `InterruptedException` is thrown. Otherwise flag is set, check with: `interrupted()` or `isInterrupted()`
Exceptions: Exceptions thrown in run() can't be propagated to the Main thread. The exception needs to be handled within the code executed on the thread.

Thread Methods Thread.x (Java 8)
start() (Start), join() (Join) wait till thread finished, timeout: join(100), sleep(ms) (Sleep) (ms) join-sleep + Timed_Waiting state, wait() gives processor-lock free, yield() scheduler can release processor, to Ready state, does not release lock, setDaemon(true) (isBackground = true);

Java Thread States: Blocked (blocked and waiting for a monitor lock), New (not yet started), Runnable (Ready to run or running), Terminated, Timed_Waiting (waiting with a specified waiting time: Thread.sleep(ms)/Thread.join(ms)), Waiting

Interrupts: Interrupted with `Thread.interrupt()` If thread is in `sleep()`, `wait()` or `join()` methods, a `InterruptedException` is thrown. Otherwise flag is set, check with: `interrupted()` or `isInterrupted()`
Exceptions: Exceptions thrown in run() can't be propagated to the Main thread. The exception needs to be handled within the code executed on the thread.

Thread Methods Thread.x (Java 8)
start() (Start), join() (Join) wait till thread finished, timeout: join(100), sleep(ms) (Sleep) (ms) join-sleep + Timed_Waiting state, wait() gives processor-lock free, yield() scheduler can release processor, to Ready state, does not release lock, setDaemon(true) (isBackground = true);

Java Thread States: Blocked (blocked and waiting for a monitor lock), New (not yet started), Runnable (Ready to run or running), Terminated, Timed_Waiting (waiting with a specified waiting time: Thread.sleep(ms)/Thread.join(ms)), Waiting

```
// java
class BankAccount {
    private int balance = 0;
    public synchronized void withdraw (int amount) {
        throws InterruptedException {
            while (amount > balance) { // not if
                await(); // wait on a condition
            }
            balance -= amount;
            // release / leave monitor
            public synchronized void deposit (int amount) {
                while (amount <= 0) {
                    wait(); // value == 0;
                }
                public synchronized void release() { value++; notify(); }
            }
        }
    }
}

// C# .NET
class BankAccount {
    private decimal balance;
    private object syncObj = new();
    public void Withdraw(decimal amount) {
        lock (syncObj) {
            while (amount > balance) {
                Monitor.Wait(syncObj);
            }
            balance -= amount;
            // release / leave monitor
            public void Deposit(decimal amount) {
                lock (syncObj) {
                    while (amount <= 0) {
                        Monitor.Wait(syncObj);
                    }
                    balance += amount;
                    notifyAll();
                }
            }
        }
    }
}
```

SPECIFIC SYNCHRONIZATION PRIMITIVES

Or has no lock and condition and no fairness flag

SEMAPHORE
Allocation of a limited number of free resources. is in essence a counter. If a resource is acquired, count--, if a resource is released, count++. Can wait until resource becomes available. Can also acquire/release multiple permits at once atomically.

```
public class Semaphore {
    private int value; public Semaphore(int initial) { value = initial; }
    public Semaphore (Semaphore s) { value = s.value; }
    public void acquire() throws InterruptedException {
        while (value <= 0) {
            wait(); // value == 0;
        }
        public synchronized void release() { value++; notify(); }
    }
}
```

General Semaphore: run Semaphore(N) Counts from 0 to N for limited permits to access a resource

Binary Semaphore: new Semaphore(1) (Counter 0 or 1 for mutual exclusion, open/closed)
Fair Semaphore: new Semaphore(N, true) (Uses FIFO queue to implement fairness. Slower than non-fair variant)
Semaphores are powerful, any synchronization can be achieved. But relatively low-level.

```
class BoundedBuffer<T> {
    private Queue<T> queue = new LinkedList<>();
    private Semaphore upperLimit = new Semaphore(capacity, true); // how many free?
    private Semaphore lowerLimit = new Semaphore(0, true); // how many full?
    public void put(T item) throws InterruptedException {
        upperLimit.acquire(); // free places - 1
        synchronized (queue) { queue.add(item); }
        lowerLimit.release(); // full places = 1
        public T get() throws InterruptedException {
            T item;
            lowerLimit.acquire(); // full places - 1
            synchronized (queue) { item = queue.remove(); }
            upperLimit.release(); // free places + 1
            return item;
        }
    }
}
```

LOCK & CONDITION

Monitor with multiple waiting lists and conditions. Independent from Monitor locks.
Lock-Object: Lock for entry in the monitor (after waiting room)
Condition-Object: Wait & Signal for a specific condition (inner waiting room)
ReentrantLock: Class in Java, alternative to synchronized. Allows multiple locking operations by the same thread and supports nested locking (Thread is able to re-enter the same lock).

Condition: Factors out the Object monitor methods (wait, notify and notifyAll) into distinct objects to give the effect of having multiple wait-sets per object, by combining them with the use of arbitrary lock implementations. A Condition replaces the use of the Object monitor methods.
condition.await(); Throws an InterruptedException if the current thread has its interrupted status set on entry to this method or is interrupted while waiting (finally frees the lock in case of interrupt), `Monitor.interrupt()`

Buffer with Lock & Condition
class BoundedBuffer<T> {
 private Queue<T> queue = new LinkedList<>();
 private Lock monitor = new ReentrantLock(true); // fair queue
 private Condition nonFull = monitor.newCondition();
 private Condition nonEmpty = monitor.newCondition();
 public void put(T item) throws InterruptedException {
 monitor.lock(); // Lock queue
 try { while (queue.size() == capacity) { nonFull.await(); }
 queue.add(item); nonEmpty.signal(); } finally { monitor.unlock(); }
 }
 public T get() throws InterruptedException {
 monitor.lock(); // wait for queue to be filled & signal to other queue
 try { while (queue.size() == 0) { nonEmpty.await(); }
 T item = queue.remove(); nonFull.signal(); return item;
 } finally { monitor.unlock(); } // always release lock, even after Exception
 }
 }
}

```
public void put(T item) throws InterruptedException {
    monitor.lock(); // Lock queue
    try { while (queue.size() == capacity) { nonFull.await(); }
        queue.add(item); nonEmpty.signal(); } finally { monitor.unlock(); }
    }
    public T get() throws InterruptedException {
        monitor.lock(); // wait for queue to be filled & signal to other queue
        try { while (queue.size() == 0) { nonEmpty.await(); }
            T item = queue.remove(); nonFull.signal(); return item;
        } finally { monitor.unlock(); } // always release lock, even after Exception
    }
}
```

	Parallel	Read	Write
Read	Yes	Yes	No
Write	No	No	No

READ-WRITE LOCK
Mutual exclusion is unnecessary for read-only threads. So one should allow parallel reading access, but implement mutual exclusion for write access.
`ReadWriteLock` `readLock = new ReentrantReadWriteLock(true);` // true for fairness
`readLock.readLock().lock();` // shared lock
// read-only accesses here
`readLock.readLock().unlock();` // release shared lock
`readLock.writeLock().lock();` // exclusive lock
// write (and read) accesses here
`readLock.writeLock().unlock();` // release exclusive lock

COUNT DOWN LATCH
Synchronization with a counter that can only count down. Threads can wait until counter ≤ 0, or they can count down. The Latches can only be used once.

CYCLIC BARRIER
Meeting point for fixed number of threads. Threads wait until everyone arrives, is reusable, threads can synchronize in multiple rounds at the same barrier (simplifies example above).

EXCHANGER
`var start = new CyclicBarrier(2); start.await();` // all cars race as they're here

Rendez-Vous: Barrier with information exchange for 2 parties. Without exchange: `new CyclicBarrier(2)`, with exchange: `Exchanger`, exchange (something). The Exchanger blocks until another thread also calls exchange(), returns argument x of the other thread.

```
var exchanger = new Exchanger<Integer>();
for (int count = 0; count < 2; count++) { // odd number of exch.: last one blocks
    new Thread(() -> {
        for (int i = 0; i < 5; i++) {
            try {
                int out = exchanger.exchange(in);
                System.out.println(Thread.currentThread().getName() + " got " + out);
            } catch (InterruptedException e) {}
        }
    }).start();
}
```

CONCURRENCY HAZARDS

DR + RC = Errorneous behaviour, **NO DR + RC = Errorneous behaviour**, **NO RC + DR = Program works correctly**, but formally incorrect, **NO RC + NO DR = Correct behaviour**

Race Conditions
Insufficiently synchronized access to shared resources. The order of events affects the correctness of the program. Leads to non-deterministic behavior. Can occur without data race, but data race is often the cause.

Race Condition without data race: critical section is not protected. Data race is eliminated using synchronization, but there is no synchronization over larger blocks, so race conditions are still possible (i.e. non-atomic incrementing).

Data Race
Two threads in a single process access the same variable concurrently without synchronization, at least one of them is a write access.

THREAD SAFETY

Dispensable cases in synchronization: Immutables (Declaring all fields private and final and don't provide setters), Read-only Objects (Read-only objects are thread-safe)

Confinement: Object belongs to only one thread at a time. **Thread Confinement** (Object belongs to one thread only, Object Confinement (Object is encapsulated in other synchronized objects)
Thread safe: A data type or method that behaves correctly when used from multiple threads as if it was running in a single thread without any additional coordination (Java concurrent collections).

Thread Safety: Avoidance of Data Races. When no sharing is intended, give each thread a private copy of the data. When sharing is important, provide explicit synchronization.

DEADLOCKS
Happens when threads lock each other out, prohibiting both from running. Programs with potential deadlock are not considered correct. Threads can suddenly block each other.

```
// Thread 1
synchronized(ListA) {
    synchronized(ListB) {
        ListB.addAll(ListA);
    }
}

// Thread 2
synchronized(ListB) {
    synchronized(ListA) {
        ListA.addAll(ListA);
    }
}
```

Both threads in this scenario have locked each other out, the program cannot continue.
LiveLocks: Threads have blocked each other permanently, but still execute wait instructions and therefore consume CPU during deadlock.

Thread 1
`b = false; while(!a) {} ... b = true;`
Thread 2
`a = false; while(!b) {} ... a = true;`

Resource Graph
Thread T waits for Lock of Resource R
Thread T acquires Lock of Resource R
Thread 1 Thread 2
account 2
account 1

Deadlocks can be identified by cycles in the resource graph.
Deadlock Avoidance: Introduce linear blocking order, lock nested only in ascending order. Or use coarse granular locks (e.g. block the whole Bank to block all accounts)

STARVATION
A thread never gets chance to access a resource. **Avoidance:** Use fair synchronization constructs. (Aging, Enable fairness in previous synchronization constructs. Monitor and Thread priorities have a fairness problem.)

PARALLELISM CORRECTNESS CRITERIA
Safety: No race conditions and no deadlocks, Liveness: No starvation
.NET SYNCHRONIZATION PRIMITIVES

Monitor with sync object: `private object sync = new(); lock(sync) { ... }`
Uses Monitor, Wait(sync), Monitor.PulseAll(sync), Uses fair FIFO-Queue. Lacks: No fairness flag, no Lock & Condition. Additional: ReadWriteLockS for Upgradable Read/Write, Semaphores can also be used at OS level, Mutex. Collections are not thread-safe.

THREAD POOLS

Thread costs: performance + memory (stack per thread) -> recycle threads for multiple tasks.
Tasks: potentially parallel work packages. Passive objects describing the functionality.
Thread Pools: tasks, smaller number of working threads grab tasks from queue and execute them. A task must run to completion before worker thread can grab a new one except nested tasks/sub-tasks.

Scalable Performance: (Rule of thumb: # of Worker Threads < # processors + 1 (Pending I/O Calls))
Programs with tasks run faster on parallel machines -> allows exploitation of parallelism without thread costs. Number of threads can be adapted to the system. **Advantages:** Thread recycling, less thread creation and releases, Higher level of abstraction (Disconnect task description from execution), Number of threads configurable on a per-system basis.

Limitations: Tasks must not wait for each other (except sub-tasks) would result in deadlock in Queue.

JAVA FORKJOINPOOL
- Get the default pool by calling `ForKJoinPool.Pool()`
- example: `int result = new CountDownLatch(2, N).invoke();`
- Default Pool (singleton), a Global shared ForkJoinPool
- used by CompletionStage
- Does not always use all processors {}

Java ForkJoinPool
Create explicit thread pool: `var threadPool = new ForkJoinPool();`
`int result = threadPool.invoke(new CountDownLatch(2, N));`
Special Features: Fire-and-forget tasks may not finish LIFO Queue per worker thread and stealing is FIFO. Automatic degree of parallelism (Default: As much worker threads as Processors), Optimized for recursive.

Work Stealing: Jobs get submitted into the global queue, which distributes the jobs to the local queues of each worker thread. If one thread has no work left, it can "steal" work from another threads local queue instead of the global queue. This distributes the scheduling work over idle processors.

- worker threads run as daemon threads workers are daemon threads in TPL.NET too.
`invoke(task):` blocking
`submit(task):` returns Future
`execute(task):` async but does not return Future fire and forget
`future.get():` wait for result/exception prevent fire and forget by evaluating result
`task.fork():` schedule async subtask
`result = task.join():` wait and get result

Task Launch
`var threadPool = new ForkJoinPool(N);`
`Future<Integer> future = threadPool.submit(() -> { // submit task into pool
 int value = ...; // long calculation
 return value;
});`
`Future<T>`
Represents a future result (asynchronous), Proxy for the result that may be not available yet because the task has not finished.

`submit():` submits task into pool and launches task
`get():` waits if necessary for computation to complete and then retrieves its result
`cancel():` Attempts to cancel execution of this task, removes it from queue Task ends when a unhandled exception occurs. It is included in the `ExecutionException` thrown by `get()`.
Fire and forget: Task is started without retrieving results later (submit() without get()). Task is run, but Exceptions will not get caught.

Count Prime Numbers

// Sequential
int counter = 0; for (int i = 2; i <= n; i++) { if (isPrime(n)) { counter++; } }

// Parallel and Recursive with RecursiveTask class:
class CountTask extends RecursiveTask<Integer> { // RecursiveAction: void function
 private final int lower, upper;
 private static final int THRESHOLD = 1; // configurable
 public CountTask(int lower, int upper) {
 this.lower = lower; this.upper = upper;
 }
 protected Integer compute() { // THRESHOLD = avoid over-parallelizing
 if (lower == upper) { return 0; }
 if (lower + 1 == upper) { return isPrime(lower) ? 1 : 0; }
 if (upper - lower < THRESHOLD) { // parallel count
 int middle = (lower + upper) / 2;
 var left = new CountTask(lower, middle);
 var right = new CountTask(middle, upper);
 return left.join() + right.join();
 }
 // V1 mit explicit fork, blockiert noch nicht left.fork(); right.fork(); // Fork = new task
 // V2 invariabel, fork launch ist sofort fertig
 invokeAll(left, right);
 return left.join() + right.join();
 }
}

`return right.join() + left.join();` // join = wait for all threads
`else { // sequential count
 int count = 0;
 for (int number = lower; number < upper; number++) {
 if (isPrime(number)) { count++; }
 }
 return count;
}`

`// invoke is blocking:
int result = new CountTask(0, N).invoke();` // invokeAll() to start multiple tasks
.NET TASK PARALLEL LIBRARY (TPL)

Preferred way to write multi-threaded and parallel code. Provides public types and APIs in System.Threading and System.Threading.Tasks namespaces. **Efficient default thread pool** (tasks are queued to the ThreadPools, supports algorithms to provide load balancing, tasks are lightweight), has multiple abstraction layers (Task Parallelization: use tasks explicitly, Data Parallelization: use parallel statements and queries using tasks implicitly, Asynchronous Programming and PLINQ).

`Task.Run(() -> { // task with return value in C#
 Task<int> task = Task.Run(() -> {
 int total = ... // some calculation
 return total;
 });
 Console.WriteLine(task.Result);` // Blocks until task is done and returns the result

`// Nested Tasks (von der Vorlesungsfolie)
var task = Task.Run(() -> {
 var left = Task.Run(() -> Count(leftPart));
 var right = Task.Run(() -> Count(rightPart));
 return left.Result + right.Result; // blockierend
});
static Task<int> Count(..., int) { ... }`

`// TDD0, CountTask von Java in C#
var left = Task.Run(() -> Count(leftPart));
var right = Task.Run(() -> Count(rightPart));
await Task.WhenAll(left, right);
return left.Result + right.Result;`

`// oder so:
class ... compute
var leftTask = Task.Run(() -> new CountTask(lower, middle).Compute());
var rightTask = Task.Run(() -> new CountTask(middle, upper).Compute());
return leftTask.Result + rightTask.Result;`

`...
int result = new CountTask(2, N).Compute();
// mit Parallel.Fork:
Parallel.Fork(2, N,
 () => 0, // thread-local count
 (i, state, localCount) => {
 // counting logic
 return localCount + 1;
 },
 localCount => {
 Interlocked.Add(ref count, localCount);
 }
);
Console.WriteLine(count);
Parallel.Invoke(2,
 Parallel.Fork(2, N,
 () => 0, // thread-local count
 (i, state, localCount) => {
 // counting logic
 return localCount + 1;
 },
 localCount => {
 Interlocked.Add(ref count, localCount);
 }
);
Console.WriteLine(count);
Parallel.Invoke(2,
 Parallel.Fork(2, N,
 () => 0, // thread-local count
 (i, state, localCount) => {
 // counting logic
 return localCount + 1;
 },
 localCount => {
 Interlocked.Add(ref count, localCount);
 }
);
Console.WriteLine(count);
Parallel.Invoke(2,
 Parallel.Fork(2, N,
 () => 0, // thread-local count
 (i, state, localCount) => {
 // counting logic
 return localCount + 1;
 },
 localCount => {
 Interlocked.Add(ref count, localCount);
 }
);
Console.WriteLine(count);
Parallel.Invoke(2,
 Parallel.Fork(2, N,
 () => 0, // thread-local count
 (i, state, localCount) => {
 // counting logic
 return localCount + 1;
 },
 localCount => {
 Interlocked.Add(ref count, localCount);
 }
);
Console.WriteLine(count);
Parallel.Invoke(2,
 Parallel.Fork(2, N,
 () => 0, // thread-local count
 (i, state, localCount) => {
 // counting logic
 return localCount + 1;
 },
 localCount => {
 Interlocked.Add(ref count, localCount);
 }
);
Console.WriteLine(count);
Parallel.Invoke(2,
 Parallel.Fork(2, N,
 () => 0, // thread-local count
 (i, state, localCount) => {
 // counting logic
 return localCount + 1;
 },
 localCount => {
 Interlocked.Add(ref count, localCount);
 }
);
Console.WriteLine(count);
Parallel.Invoke(2,
 Parallel.Fork(2, N,
 () => 0, // thread-local count
 (i, state, localCount) => {
 // counting logic
 return localCount + 1;
 },
 localCount => {
 Interlocked.Add(ref count, localCount);
 }
);
Console.WriteLine(count);
Parallel.Invoke(2,
 Parallel.Fork(2, N,
 () => 0, // thread-local count
 (i, state, localCount) => {
 // counting logic
 return localCount + 1;
 },
 localCount => {
 Interlocked.Add(ref count, localCount);
 }
);
Console.WriteLine(count);
Parallel.Invoke(2,
 Parallel.Fork(2, N,
 () => 0, // thread-local count
 (i, state, localCount) => {
 // counting logic
 return localCount + 1;
 },
 localCount => {
 Interlocked.Add(ref count, localCount);
 }
);
Console.WriteLine(count);
Parallel.Invoke(2,
 Parallel.Fork(2, N,
 () => 0, // thread-local count
 (i, state, localCount) => {
 // counting logic
 return localCount + 1;
 },
 localCount => {
 Interlocked.Add(ref count, localCount);
 }
);
Console.WriteLine(count);
Parallel.Invoke(2,
 Parallel.Fork(2, N,
 () => 0, // thread-local count
 (i, state, localCount) => {
 // counting logic
 return localCount + 1;
 },
 localCount => {
 Interlocked.Add(ref count, localCount);
 }
);
Console.WriteLine(count);
Parallel.Invoke(2,
 Parallel.Fork(2, N,
 () => 0, // thread-local count
 (i, state, localCount) => {
 // counting logic
 return localCount + 1;
 },
 localCount => {
 Interlocked.Add(ref count, localCount);
 }
);
Console.WriteLine(count);
Parallel.Invoke(2,
 Parallel.Fork(2, N,
 () => 0, // thread-local count
 (i, state, localCount) => {
 // counting logic
 return localCount + 1;
 },
 localCount => {
 Interlocked.Add(ref count, localCount);
 }
);
Console.WriteLine(count);
Parallel.Invoke(2,
 Parallel.Fork(2, N,
 () => 0, // thread-local count
 (i, state, localCount) => {
 // counting logic
 return localCount + 1;
 },
 localCount => {
 Interlocked.Add(ref count, localCount);
 }
);
Console.WriteLine(count);
Parallel.Invoke(2,
 Parallel.Fork(2, N,
 () => 0, // thread-local count
 (i, state, localCount) => {
 // counting logic
 return localCount + 1;
 },
 localCount => {
 Interlocked.Add(ref count, localCount);
 }
);
Console.WriteLine(count);
Parallel.Invoke(2,
 Parallel.Fork(2, N,
 () => 0, // thread-local count
 (i, state, localCount) => {
 // counting logic
 return localCount + 1;
 },
 localCount => {
 Interlocked.Add(ref count, localCount);
 }
);
Console.WriteLine(count);
Parallel.Invoke(2,
 Parallel.Fork(2, N,
 () => 0, // thread-local count
 (i, state, localCount) => {
 // counting logic
 return localCount + 1;
 },
 localCount => {
 Interlocked.Add(ref count, localCount);
 }
);
Console.WriteLine(count);
Parallel.Invoke(2,
 Parallel.Fork(2, N,
 () => 0, // thread-local count
 (i, state, localCount) => {
 // counting logic
 return localCount + 1;
 },
 localCount => {
 Interlocked.Add(ref count, localCount);
 }
);
Console.WriteLine(count);
Parallel.Invoke(2,
 Parallel.Fork(2, N,
 () => 0, // thread-local count
 (i, state, localCount) => {
 // counting logic
 return localCount + 1;
 },
 localCount => {
 Interlocked.Add(ref count, localCount);
 }
);
Console.WriteLine(count);
Parallel.Invoke(2,
 Parallel.Fork(2, N,
 () => 0, // thread-local count
 (i, state, localCount) => {
 // counting logic
 return localCount + 1;
 },
 localCount => {
 Interlocked.Add(ref count, localCount);
 }
);
Console.WriteLine(count);
Parallel.Invoke(2,
 Parallel.Fork(2, N,
 () => 0, // thread-local count
 (i, state, localCount) => {
 // counting logic
 return localCount`

```
// CPU
volatile bool a = false, b = false;
// Thread 1
a = true; while( !b ) { ... }
// Thread 2
b = true; while( !a ) { ... }

// GPU
volatile bool a = false, b = false;
// Thread 1
a = true; Thread.MemoryBarrier();
while( !b ) { ... }
// Thread 2
b = true; Thread.MemoryBarrier();
while( !a ) { ... }
```

Spin-Lock with Atomic Operation:

```
public class SpinLock {
    private final AtomicBoolean locked = new AtomicBoolean(false); // unlocked
    public void acquire() { while( locked.getAndSet(true) ) { ... } }
    public void release() { locked.set(false); }
```

Java Atomic Classes: Classes for boolean, Integer, Long, References and Array-Elements. Different kinds of atomic operations, `addAndGet()`, `getAndAdd()` etc.

Operations on atomic data classes: `boolean getAndSet(boolean newValue)`

Atomically sets to the given value and returns the previous value.

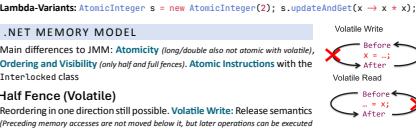
`boolean compareAndSet(boolean expected, boolean update)`

Sets update only when read value is equal to expect. Returns true when successful.

Optimistic Synchronization: *(Read old value and then compare before writing if value is still the same. If not, retry)*

```
do { old = v.get(); new = result; } while( !v.compareAndSet(old, new));
```

Lambda-Variables: `AtomicInteger x = new AtomicInteger(2); s.updateAndGet(x -> x + x);`



Full Fence (Memory Barrier)

Disallows reordering in both directions. `Thread.MemoryBarrier();`

GPU (GRAPHICS PROCESSING UNIT)

End of Moore's Law: We can no longer gain performance by "growing" sequential processors. Instead, we improve performance by running code in parallel on multi-core (CPUs) (low latency) and many-core or massively parallel co-processors (GPUs) (high throughput).

GPUs are specialized electronic circuits designed to accelerate the computation of computer graphics. They are faster than CPUs for solvable algorithms on large datasets. Useful for calculations which consist of multiple independent sub-calculations, not very useful for calculations where the results rely on the previous results (like Fibonacci).

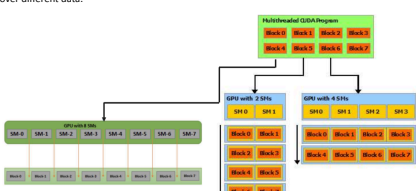
High Parallelization: A CPU offers few cores (4, 8, 16, 64) and is very fast. Programming is easier. A GPU offers a very large number of cores (512, 1024, 3584, 5760) and has very specific slower processors. It is optimized for throughput. Programming is more difficult.

GPU Structure: A GPU consists of multiple **Streaming Multiprocessors (SM)** which in turn consist of multiple **Streaming Processors (SP)** (e.g. 30 SM x 102 SP per SM).

SIMD: Multiple threads execute the same instruction is executed simultaneously on multiple cores working on different data elements (*vector parallelism*). Saves fetch & decode instructions.

SISD: Single Instruction Single Data. Purely sequential calculations.

SIMT: Single Instruction Multiple Threads. The same instruction is executed in different threads over different data.



PERFORMANCE SCALING

Difficulties with parallel programs: Finding parallelism, granularity of a parallel task, moving data is expensive, load balancing, coordination & synchronization, performance debugging.

Scalability: The ability of hard- and software to deliver **greater computational power** when the number of resources is increased.

Scalability Testing: Primary challenge of parallel computing is **deciding how best to break up a problem** into individual pieces that can be computed separately. It is **impractical** to develop and test large applications using the **full problem size**. The problem and number of processors are **scaled down** at first. **Scalability testing:** measuring the ability of an application to perform well or better with varying problem sizes and numbers of processors. It does **not** test the applications **general functionality** or correctness.

STRONG SCALING

The number of processors is **increased while the problem size remains constant**. Results in a reduced workload per processor. Mostly used for long running CPU bound applications.

Amdahls Law: The speedup is **limited by the fraction of the serial part** of the software that is not amenable to parallelization. Sweet spot needs to be found. It is reasonable to use **small amounts of resources for small problems and larger quantities of resources for big problems**.

$T =$ total time, $p =$ part of the program that can be parallelized,, $N =$ amount of processors

$T = (1 - p)T + T_p = T_s + T_p$

$T_N = T_p/N + (1 - p)T$, Speedup $\leq 1/(s + p/N)$, Efficiency $= T/(N \cdot T_N)$

Amdahls law **ignores the parallel overhead**. Because of that, it is **the upper limit** of speedup for a problem of fixed size. This seems to be a **bottleneck** for parallel computing.

Examples: 90% of the computation can run parallel, what is the max speedup with 8 processors?

$1/(0.1 + 0.9/8) \approx 4.7$

25% of the computation must be serial. What is the max speedup with ∞ Processors?

$1/(0.25 + 0.75/\infty) \approx 1/0.25 = 4$

To gain $500 \times$ speedup on 1000 processors, Amdahls law requires that the proportion of serial part cannot exceed what?

$500 = 1/(s + \frac{1-p}{1000}) \Rightarrow s + \frac{1-p}{1000} = \frac{1}{500} \Rightarrow 1000s + (1 - s) = 2 \Rightarrow 999s = 1 \Rightarrow s = \frac{1}{999} \approx 0.1\%$

WEAK SCALING

The number of processors and the problem size is **increased**. Mostly used for large **memory-bound** applications where the required memory cannot be satisfied by a single node.

Gustafson's law: Based on the approximations that the **parallel part scales linearly** with the amount of resources, and that the **serial part does not increase** with respect to the size of the problem. Speedup $= s + p \cdot N = s + (1 - s) \cdot N$

In this case, the problem size assigned to each processing element stays **constant** and **additional elements** are used to solve a **larger total problem**. Therefore, this type of measurement is justification for programs that take a lot of memory or other system resources.

Example: 64 Processors. 5% of the program is serial, What is the scaled weak speedup?

$0.05 + 0.95 \cdot 64 = \underline{60.85}$

ParProg | FS24 | Nina Grössli & Jannis Tschan

Page 3